Precondicionamiento y gradiente conjugado precondicionado

Clase 11 · Capítulo 5 · Sección 5.1.6

Análisis Numérico (LMA) · Métodos Numéricos II (LCD)

22 de septiembre de 2026

Facultad de Ingeniería Química · UNL

Dónde quedamos

Teorema 5.15 (cota de Tchebychev)

$$\|x_k - x_*\|_A \leq 2\left(\frac{\sqrt{\kappa_2(A)}-1}{\sqrt{\kappa_2(A)}+1}\right)^k \|x_0 - x_*\|_A, \qquad k \geq 0.$$

Y un algoritmo barato: 1 mat-vec, 2 productos escalares, 3 *axpy*, 4 vectores.

Pero: si $\kappa_2(A)$ es grande, la convergencia es lenta igual. Con $\kappa = 10^4$ el factor es ≈ 0.980 por iteración.

Y κ grande es *la regla*, no la excepción: al discretizar una EDP con paso h, típicamente $\kappa_2(A) = O(h^{-2})$.

La idea de esta clase

Si no podemos cambiar el método, cambiemos el sistema: resolvamos otro sistema, equivalente, pero mejor condicionado.

Plan de la clase

1. **La idea de precondicionar:** $Ax = b \iff MAx = Mb$ con $M \approx A^{-1}$.
2. **El obstáculo:** MA no es simétrica. La solución teórica: $S = M^{1/2}$ y el sistema $SASy = Sb$.
3. **Deducción del GCP:** reescribir el algoritmo en términos de $x_k = Sy_k$ hasta que S *desaparezca*.
4. **El algoritmo del GCP** y su criterio de parada.
5. **Qué es un buen precondicionador:** las dos condiciones en tensión.
6. **Precondicionadores genéricos:** Jacobi, Gauss–Seidel simétrico, factorización incompleta.

5.1.6 La idea del precondicionamiento

Cambiar el sistema, no el método

Sea $M \in \mathbb{R}^{n \times n}$ invertible. Entonces

$$Ax = b \qquad \Longleftrightarrow \qquad MAx = Mb.$$

Los dos sistemas tienen la misma solución, pero distinto condicionamiento.

Si $M \approx A^{-1}$, entonces $MA \approx I$, y $\kappa(MA) \approx 1$: un método iterativo aplicado a $MAx = Mb$ debería converger mucho más rápido.

Dos casos extremos

- $M = I$: aplicar M es gratis, pero no ganamos nada.
- $M = A^{-1}$: $\kappa(MA) = 1$ (¡una iteración!), pero aplicar M es tan caro como resolver el problema original.

Todo el juego está en el equilibrio entre estas dos cosas.

$M \approx A^{-1}$: ¿en qué sentido?

Definición

Sea A sdp. Decimos que M sdp es un precondicionador de A si existen constantes $0 < c_1 \leq c_2$ tales que

$$c_1 \leq \frac{x^T M x}{x^T A^{-1} x} \leq c_2, \qquad \forall\, x \neq 0.$$

Si $\frac{c_2}{c_1}$ es cercano a 1, decimos que M es un buen precondicionador.

Lectura: las formas cuadráticas de M y de A^{-1} son *equivalentes*, con constantes c_1, c_2. No pedimos $M \approx A^{-1}$ entrada por entrada: pedimos que actúen parecido.

Notar que $M = A^{-1}$ da $c_1 = c_2 = 1$, y $M = I$ da $c_1 = \lambda_{\min}(A)$, $c_2 = \lambda_{\max}(A)$, o sea $\frac{c_2}{c_1} = \kappa_2(A)$: no precondicionar es el peor caso.

El obstáculo: MA no es simétrica

Problema

Aunque A y M sean ambas sdp, el producto MA en general no es simétrico.

Luego no podemos aplicar el gradiente conjugado al sistema $MAx = Mb$.

La salida: la raíz cuadrada

Sea S la raíz cuadrada de M: la única matriz sdp con $S^2 = M$. (Existe: si $M = Q\Lambda Q^T$ con $\Lambda > 0$, tomar $S = Q\Lambda^{1/2}Q^T$.)

Como S es invertible,

$$Ax = b \iff MAx = Mb \iff S^2Ax = S^2b \iff SAx = Sb.$$

E insertando $I = SS^{-1}$ y llamando $y := S^{-1}x$:

$$Ax = b \iff \begin{cases} SAS\,y = Sb \\ x = Sy \end{cases}$$

El sistema transformado

Ejercicio 1

Verificar que SAS es sdp.

(Simetría: $(SAS)^T = S^T A^T S^T = SAS$. Definida positiva: $z^T SASz = (Sz)^T A(Sz) > 0$ si $z \neq 0$, porque S es invertible.)

Ejercicio 2 (clave)

Verificar que

$$\kappa_2(SAS) \leq \frac{c_2}{c_1}.$$

Sugerencia: $\sigma(SAS) = \sigma(MA)$ y acotar el cociente de Rayleigh $\dfrac{z^T SASz}{z^T z}$ con la definición de precondicionador, tomando $x = S^{-1}z$.

Moraleja: el GC aplicado a $SASy = Sb$ converge con factor $\left(\dfrac{\sqrt{\frac{c_2}{c_1}} - 1}{\sqrt{\frac{c_2}{c_1}} + 1}\right)^k$: $\frac{c_2}{c_1}$ reemplaza a $\kappa_2(A)$.

Recordatorio: el GC de la clase pasada

Gradiente conjugado

Sean A sdp, $b \in \mathbb{R}^n$, y $x \in \mathbb{R}^n$ (iterando inicial), $\mathrm{tol} \in \mathbb{R}_+$, $\texttt{maxiter} \in \mathbb{N}$.

$r = b - Ax$; $\rho = r^T r$; $\mathrm{TOL} = \mathrm{tol}^2\, b^T b$; $k = 0$; % $\rho = \|r\|_2^2$, $\mathrm{TOL} = \mathrm{tol}^2 \|b\|_2^2$

mientras ($\rho > \mathrm{TOL}$ **y** $k <$ `maxiter`) % $\|r\|_2/\|b\|_2 > \mathrm{tol}$

  si $k = 0$: $d = r$; **si no**: $\beta = \rho/\rho_{old}$; $d = r + \beta d$;

  $w = Ad$; $\alpha = \rho/(d^T w)$; $x = x + \alpha d$;

  $r = r - \alpha w$; % $r_{k+1} = r_k - \alpha_{k+1} A d_{k+1}$

  $\rho_{old} = \rho$; $\rho = r^T r$; $k = k + 1$;

fin

Por iteración: 1 mat-vec, 2 productos escalares, 3 *axpy*. Cuatro vectores en memoria: x, r, d, w.

Este es el algoritmo que vamos a aplicar a $SASy = Sb$, y el que al final queremos recuperar *sin* S.

El plan

1. Aplicar el GC (el de la clase pasada, tal cual) al sistema $SASy = Sb$. Llamamos y_k, $\tilde{r}_k$, $\tilde{d}_k$ a los iterados, residuos y direcciones.
2. Definir $x_k := Sy_k$ y reescribir todo el algoritmo en términos de x_k, $r_k := b - Ax_k$, d_k.
3. Observar que en las fórmulas resultantes S aparece siempre al cuadrado: solo aparece $M = S^2$.

Por qué importa

$S = M^{1/2}$ no la conocemos y no la queremos calcular. Es solo una herramienta teórica: al final del cálculo desaparece.

GC aplicado a $SASy = Sb$

Gradiente conjugado en la variable y

Sea $y_0 \in \mathbb{R}^n$. $\tilde{r}_0 = Sb - SASy_0$; $\tilde{\rho}_0 = \|\tilde{r}_0\|_2^2$; $k = 0$;

mientras $\tilde{\rho}_k \neq 0$

si $k = 0$: $\tilde{d}_1 = \tilde{r}_0$; **si no**: $\tilde{\beta}_{k+1} = \dfrac{\tilde{\rho}_k}{\tilde{\rho}_{k-1}}$, $\tilde{d}_{k+1} = \tilde{r}_k + \tilde{\beta}_{k+1}\tilde{d}_k$;

$\tilde{w}_{k+1} = SAS\,\tilde{d}_{k+1}$; $\tilde{\alpha}_{k+1} = \dfrac{\tilde{\rho}_k}{\tilde{d}_{k+1}^T \tilde{w}_{k+1}}$;

$y_{k+1} = y_k + \tilde{\alpha}_{k+1}\tilde{d}_{k+1}$; $\tilde{r}_{k+1} = \tilde{r}_k - \tilde{\alpha}_{k+1}\tilde{w}_{k+1}$;

$\tilde{\rho}_{k+1} = \|\tilde{r}_{k+1}\|_2^2$; $k = k + 1$;

fin

Es exactamente el algoritmo de la clase pasada, con $A \rightsquigarrow SAS$ y $b \rightsquigarrow Sb$.

Tal como está, no es implementable: pide S.

La cota de error vuelve a la variable original

Sea y_* la solución de $SASy = Sb$ y $x_* = Sy_*$. Como $\kappa_2(SAS) \leq \frac{c_2}{c_1}$,

$$\|y_k - y_*\|_{SAS} \leq 2 \left(\frac{\sqrt{\frac{c_2}{c_1}} - 1}{\sqrt{\frac{c_2}{c_1}} + 1} \right)^k \|y_0 - y_*\|_{SAS}.$$

El puente entre las dos normas

Para todo $z \in \mathbb{R}^n$:

$$\|z\|_{SAS}^2 = z^T SASz = (Sz)^T A(Sz) = \|Sz\|_A^2.$$

Aplicándolo a $z = y_k - y_*$ (y notando $S(y_k - y_*) = x_k - x_*$):

$$\boxed{\|x_k - x_*\|_A \leq 2 \left(\frac{\sqrt{\frac{c_2}{c_1}} - 1}{\sqrt{\frac{c_2}{c_1}} + 1} \right)^k \|x_0 - x_*\|_A, \qquad k \geq 0.}$$

Misma norma, misma forma de cota que antes, pero con $\frac{c_2}{c_1}$ en lugar de $\kappa_2(A)$.

5.1.6 Deducción del algoritmo: que S desaparezca

Traducción de los vectores

Definimos $x_k := Sy_k$ y $r_k := b - Ax_k$.

Residuos

$$r_k = b - Ax_k = b - ASy_k = S^{-1}\big(Sb - SASy_k\big) = S^{-1}\tilde{r}_k \qquad \Longleftrightarrow \qquad \tilde{r}_k = Sr_k.$$

Direcciones

De $y_{k+1} = y_k + \tilde{\alpha}_{k+1}\tilde{d}_{k+1}$, multiplicando por S:

$$x_{k+1} = x_k + \tilde{\alpha}_{k+1}\, d_{k+1}, \qquad \text{con} \quad d_{k+1} := S\tilde{d}_{k+1}.$$

Observación

El paso en x tiene *la misma* longitud $\tilde{\alpha}_{k+1}$ que el paso en y: solo cambia la dirección.

La recurrencia de las direcciones

Multiplicamos por S la recurrencia $\tilde{d}_{k+1} = \tilde{r}_k + \tilde{\beta}_{k+1}\tilde{d}_k$:

$$d_1 = S\tilde{d}_1 = S\tilde{r}_0 = S\,(Sr_0) = S^2 r_0 = Mr_0,$$

$$d_{k+1} = S\tilde{d}_{k+1} = S\tilde{r}_k + \tilde{\beta}_{k+1}S\tilde{d}_k = S(Sr_k) + \tilde{\beta}_{k+1}d_k = Mr_k + \tilde{\beta}_{k+1}d_k, \qquad k \geq 1.$$

Primera aparición de M

Cada vez que S aparece, aparece dos veces: $S \cdot S = M$.

La dirección de búsqueda ya no es el residuo, sino el residuo precondicionado $z_k := Mr_k$.

Comparar con el GC sin precondicionar: allí $d_1 = r_0$ y $d_{k+1} = r_k + \beta_{k+1}d_k$. Es el caso $M = I$.

Los coeficientes

Para $\tilde{\alpha}_{k+1}$: usando $\tilde{r}_k = Sr_k$ y $\tilde{d}_{k+1} = S^{-1}d_{k+1}$,

$$\tilde{\alpha}_{k+1} = \frac{\tilde{r}_k^T \tilde{r}_k}{\tilde{d}_{k+1}^T SAS\, \tilde{d}_{k+1}} = \frac{(Sr_k)^T (Sr_k)}{(S\tilde{d}_{k+1})^T A (S\tilde{d}_{k+1})} = \frac{r_k^T S S r_k}{d_{k+1}^T A d_{k+1}} = \frac{r_k^T M r_k}{d_{k+1}^T A d_{k+1}}.$$

Para $\tilde{\beta}_{k+1}$:

$$\tilde{\beta}_{k+1} = \frac{\tilde{r}_k^T \tilde{r}_k}{\tilde{r}_{k-1}^T \tilde{r}_{k-1}} = \frac{(Sr_k)^T (Sr_k)}{(Sr_{k-1})^T (Sr_{k-1})} = \frac{r_k^T M r_k}{r_{k-1}^T M r_{k-1}}.$$

S desapareció

En *todas* las fórmulas S aparece de a pares: solo hace falta la acción $\xi \mapsto M\xi$. Con $\rho_k := r_k^T M r_k = \|r_k\|_M^2$ queda $\tilde{\beta}_{k+1} = \dfrac{\rho_k}{\rho_{k-1}}$, $\tilde{\alpha}_{k+1} = \dfrac{\rho_k}{d_{k+1}^T A d_{k+1}}$: *idénticas* a las del GC, con $\|\cdot\|_2$ reemplazada por $\|\cdot\|_M$.

Criterio de parada

En la variable y vale la estimación habitual

$$\frac{\|y_k - y_*\|_2}{\|y_*\|_2} \;\leq\; \kappa_2(SAS)\,\frac{\|\tilde{r}_k\|_2}{\|Sb\|_2}.$$

Como $\tilde{r}_k = Sr_k$ y $\|Sv\|_2^2 = v^T S^2 v = \|v\|_M^2$, y $\|y_k - y_*\|_2 = \|S^{-1}(x_k - x_*)\|_2 = \|x_k - x_*\|_{M^{-1}}$, esto se traduce en

$$\frac{\|x_k - x_*\|_{M^{-1}}}{\|x_*\|_{M^{-1}}} \;\leq\; \kappa_2(SAS)\,\frac{\|r_k\|_M}{\|b\|_M}.$$

Criterio usual

Dada tol, se itera hasta que

$$\frac{\|r_k\|_M}{\|b\|_M} < \mathrm{tol} \qquad \Longleftrightarrow \qquad \rho_k < \mathrm{tol}^2\, b^T M b.$$

(De nuevo se comparan cuadrados: $\rho_k = r_k^T M r_k$ ya se calcula en cada paso.)

Ojo: igual que antes, el factor $\kappa_2(SAS)$ está ahí. Conviene también un tope `maxiter`.

El algoritmo del gradiente conjugado precondicionado

GCP

Sean A sdp, $b \in \mathbb{R}^n$, y M sdp (precondicionador, $M \approx A^{-1}$), $x \in \mathbb{R}^n$, $\mathrm{tol} \in \mathbb{R}_+$, $\texttt{maxiter} \in \mathbb{N}$.

$r = b - Ax$; $z = Mr$; $\rho = r^T z$; $\mathrm{TOL} = \mathrm{tol}^2\, b^T M b$; $k = 0$; % $\rho = \|r\|_M^2$

mientras ($\rho > \mathrm{TOL}$ **y** $k < \texttt{maxiter}$) % $\|r\|_M / \|b\|_M > \mathrm{tol}$

  si $k = 0$: $d = z$; **si no**: $\beta = \rho / \rho_{old}$; $d = z + \beta d$; % $d = Mr$

  $w = Ad$; $\alpha = \rho / (d^T w)$; $x = x + \alpha d$;

  $r = r - \alpha w$; $z = Mr$; % la única línea nueva

  $\rho_{old} = \rho$; $\rho = r^T z$; $k = k + 1$;

fin

Cinco vectores en memoria: x, r, z, d, w. Uno más que el GC.

GC vs. GCP, lado a lado

	GC	**GCP**
Inicialización	$r = b - Ax;\ \ \rho = r^T r$	$r = b - Ax;\ \ z = Mr;\ \ \rho = r^T z$
Dirección	$d = r + \beta d$	$d = z + \beta d$
Actualizar r	$r = r - \alpha w$	$r = r - \alpha w;\ \ z = Mr$
Escalar	$\rho = r^T r = \|r\|_2^2$	$\rho = r^T z = \|r\|_M^2$
Parada	$\|r\|_2 < \mathrm{tol}\|b\|_2$	$\|r\|_M < \mathrm{tol}\|b\|_M$
Memoria	4 vectores	5 vectores
Costo/iteración	1 mat-vec (Ad)	1 mat-vec + 1 acción Mr
Tasa	$\left(\frac{\sqrt{\kappa_2(A)}-1}{\sqrt{\kappa_2(A)}+1}\right)^k$	$\left(\frac{\sqrt{c_2/c_1}-1}{\sqrt{c_2/c_1}+1}\right)^k$

El GC es el caso $M = I$: ahí $z = r$ y todo colapsa al algoritmo de la clase pasada.

Observaciones

1. **No hace falta calcular $S = M^{1/2}$.** Fue solo una herramienta teórica para la deducción; en el algoritmo final no aparece.
2. **No hace falta conocer M como matriz.** Alcanza con una rutina que calcule $\xi \mapsto M\xi$. (Igual que con A: alcanza con $d \mapsto Ad$.)
3. **El sobrecosto es una sola acción $M\xi$ por iteración**, más un vector en memoria. Si esa acción es cara, el precondicionador no sirve aunque reduzca mucho $\frac{c_2}{c_1}$.
4. **La cota de error está en norma A**, igual que antes:

$$\|x_k - x_*\|_A \leq 2\left(\frac{\sqrt{c_2/c_1}-1}{\sqrt{c_2/c_1}+1}\right)^k \|x_0 - x_*\|_A.$$

Implementación en matlab/octave

```
function [x, k, resrel] = gcp(A, b, M, x, tol, maxiter)
% Gradiente conjugado precondicionado.  A sdp, M sdp aprox. de inv(A).
% A y M pueden ser matrices o function handles (acciones v -> A*v, v -> M*v).
  if isa(A,'function_handle'), Av = A; else, Av = @(v) A*v; end
  if isa(M,'function_handle'), Mv = M; else, Mv = @(v) M*v; end
  r = b - Av(x);   z = Mv(r);   rho = r'*z;
  TOL = tol^2*(b'*Mv(b));   k = 0;   d = 0*x;
  while rho > TOL && k < maxiter
    if k == 0,  d = z;
    else        d = z + (rho/rho_old)*d;       % beta = rho/rho_old
    end
    w     = Av(d);                             % unico producto matriz-vector
    alpha = rho/(d'*w);
    x = x + alpha*d;
    r = r - alpha*w;
    z = Mv(r);                                 % accion del precondicionador
    rho_old = rho;   rho = r'*z;   k = k + 1;
  end
  resrel = sqrt(rho/(b'*Mv(b)));
end
```

Por iteración: 1 mat-vec, 1 acción $M\xi$, 2 productos escalares, 3 *axpy*.

La función `pcg` de matlab/octave

Atención al convenio

Para MATLAB/OCTAVE, el precondicionador M es una aproximación de A, no de A^{-1}.

Con ese convenio, las dos condiciones de un buen precondicionador se leen:

- Es "fácil" resolver sistemas con matriz M (o sea, la acción $\xi \mapsto M^{-1}\xi$ es barata).
- $M \approx A$ en el sentido: existen c_1, c_2 con

$$c_1 \leq \frac{x^T M x}{x^T A x} \leq c_2 \quad \forall x \neq 0, \qquad \text{con } \tfrac{c_2}{c_1} \approx 1.$$

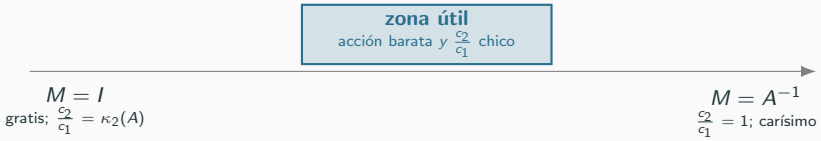
Es el mismo objeto con el nombre cambiado: nuestro M es el M^{-1} de MATLAB/OCTAVE. Uso típico: `x = pcg(A,b,tol,maxit,M1,M2)`, donde $M \approx M_1 M_2$ (p. ej. $M_1 = R^T$, $M_2 = R$ de una factorización incompleta).

5.1.6 ¿Qué es un buen precondicionador?

Las dos condiciones, en tensión

Un buen precondicionador de A sdp es M sdp tal que

1. la acción $\xi \mapsto M\xi$ es barata de calcular;
2. $M \approx A^{-1}$: existen c_1, c_2 con $c_1 \leq \dfrac{x^T M x}{x^T A^{-1} x} \leq c_2$ y $\frac{c_2}{c_1} \approx 1$.



La moraleja práctica: para construir un buen M hay que saber de dónde viene A (típicamente: de la discretización de una ecuación diferencial) y usar esa información.

5.1.6 Precondicionadores genéricos

Precondicionadores genéricos

Son los que se construyen para cualquier A, sin mirar su origen.

Expectativa realista

Con precondicionadores genéricos no hay que esperar un cambio *dramático* en la convergencia. Mejoran la constante, rara vez el orden.

La receta: reciclar un método estacionario

Si M es invertible,

$$Ax = b \iff MAx = Mb \iff x = (I - MA)x + Mb,$$

y la iteración $x_{k+1} = (I - MA)x_k + Mb$ converge sii $\rho(I - MA) < 1$.

Idea: elegir M tal que $I - MA = B$, con B la matriz de iteración de un método estacionario conocido (Jacobi, Gauss–Seidel, ...).

Notación (Cap. 4): $A = L + D + U$, con D diagonal, L triangular inferior estricta, U triangular superior estricta.

Precondicionamiento diagonal (Jacobi)

La matriz de iteración de Jacobi es $B_J = -D^{-1}(L+U)$. Buscamos M_J con $I - M_J A = B_J$:

$$M_J A = I + D^{-1}(L+U) = D^{-1}\underbrace{(D+L+U)}_{=A} \qquad \Longrightarrow \qquad \boxed{M_J = D^{-1}.}$$

- La acción $\xi \mapsto D^{-1}\xi$ es trivial: n divisiones.
- Es sdp si A es sdp (los $a_{ii} > 0$).
- Equivale a un *reescalado* de filas y columnas: arregla los problemas de condicionamiento que vienen de tener unidades o escalas dispares en las incógnitas.
- Si A ya tiene diagonal constante (p. ej. el laplaciano discreto), $M_J = \frac{1}{a} I$: no cambia nada.

Regla práctica: es tan barato que casi siempre conviene probarlo primero.

Gauss–Seidel simétrico

¿Por qué "simétrico"? Gauss–Seidel solo no sirve: $B_{GS} = -(L+D)^{-1}U$ da un M no simétrico, y el GCP necesita M sdp.

Se combina un barrido hacia adelante con uno hacia atrás:

$$B_{GS} = -(L+D)^{-1}U, \qquad B_{BGS} = -(U+D)^{-1}L, \qquad B_{SGS} = B_{BGS}B_{GS} = (U+D)^{-1}L(L+D)^{-1}U.$$

Si A es simétrica, $U = L^T$ y $B_{SGS} = (L^T+D)^{-1}L(L+D)^{-1}L^T$. El precondicionador es

$$\boxed{M_{SGS} = (L^T+D)^{-1}\,D\,(L+D)^{-1}}$$

Ejercicio

Verificar que $I - M_{SGS}A = B_{SGS}$. (Cuenta guiada en el apunte; la clave es $L = L^T\big((L+D)^{-1}L^T\big)^{-1} - D$.)

La acción $\xi \mapsto M_{SGS}\xi$

(i) resolver un sistema triangular inferior; (ii) multiplicar por una diagonal; (iii) resolver un sistema triangular superior. Sin factorizar nada: L y D ya están en A. Y M_{SGS} es sdp por construcción (de la forma C^TDC).

Factorización incompleta

Motivación: Cholesky $A = R^T R$ resolvería el sistema de una, pero para A rala R sufre fill-in: se llena de entradas no nulas.

Idea: hacer Cholesky *descartando* el fill-in (p. ej. forzando el patrón de ceros de A). Se obtiene R triangular superior rala con

$$A = R^T R + E, \qquad E \text{ “chica”}.$$

Se toma entonces

$$\boxed{M = (R^T R)^{-1} = R^{-1}(R^T)^{-1}}$$

y la acción $\xi \mapsto M\xi$ son dos sustituciones: una hacia adelante con R^T y una hacia atrás con R.

- M es sdp (de la forma $C^T C$ con $C = (R^T)^{-1}$).
- Costo: la factorización se calcula una sola vez, antes de iterar.
- En MATLAB/OCTAVE: `cholinc` (hoy `ichol`) para matrices sdp; `luinc`/`ilu` en el caso general.

Comparación

Precondicionador	M	**Acción** $\xi \mapsto M\xi$	**Setup**
Ninguno	I	gratis	—
Diagonal (Jacobi)	D^{-1}	n divisiones	—
Gauss–Seidel sim.	$(L^T + D)^{-1} D (L + D)^{-1}$	2 sustituciones + diagonal	—
Cholesky incompleta	$(R^T R)^{-1}$	2 sustituciones	factorizar una vez
Basado en la estructura	*(multigrid, ...)*	$O(n)$	*depende*

Lo que falta

Los genéricos mejoran constantes. Para que $\frac{c_2}{c_1}$ quede acotado independientemente de n hay que usar el origen de A: es la sección 5.1.7, multigrid.

Para recordar

1. **Precondicionar** = resolver $MAx = Mb$ con $M \approx A^{-1}$, para que el condicionamiento efectivo baje.
2. MA no es simétrica $\Rightarrow$ el truco teórico $S = M^{1/2}$ y el sistema $SASy = Sb$, que sí es sdp, con $\kappa_2(SAS) \leq \frac{c_2}{c_1}$.
3. Al reescribir el GC en la variable $x_k = Sy_k$, S desaparece: solo aparece $M = S^2$.
4. **GCP = GC con dos cambios:** la dirección usa el residuo precondicionado $z = Mr$, y los escalares usan $\rho = r^T z = \|r\|_M^2$.
5. **Cota:** $\|x_k - x_*\|_A \leq 2\Big(\dfrac{\sqrt{c_2/c_1} - 1}{\sqrt{c_2/c_1} + 1}\Big)^k \|x_0 - x_*\|_A$.
6. **Buen precondicionador:** acción barata y $\frac{c_2}{c_1} \approx 1$. Los extremos $M = I$ y $M = A^{-1}$ fallan uno en cada condición.
7. **Genéricos:** $M_J = D^{-1}$, $M_{SGS} = (L^T + D)^{-1} D (L + D)^{-1}$, $M = (R^T R)^{-1}$ (Cholesky incompleta). Mejoras modestas.
8. **Próxima:** precondicionadores que usan la estructura del problema: multigrid en 1D (Sección 5.1.7).